I am creating APIs for a Mobility Solution company. This company offer many services including vehicle valuation. I have come up with a one core system which bring together all those service, where I am using micro-services to represent each service. The programming technologies used include :-

.Backend: Go 1.21, Gin HTTP framework

- **Database**: PostgreSQL with GORM
- **Cache**: Redis
- **Message Queue**: Apache Kafka
- **Authentication**: JWT tokens
- **Storage**: Firebase Storage
- **Notifications**: Firebase Cloud Messaging
- **Documentation**: Swagger/OpenAPI
- **Containerization**: Docker

The project micro-service structure is as follow. The below is user-management micro-service.

```
internal/
├── vehicle-service/
│   ├── handlers/       # HTTP handlers
│   ├── models/         # Data models
│   ├── repositories/   # Data access layer
│   ├── services/       # Business logic
│   └── routes/         # Route definitions
├── shared/
│   ├── database/       # Database connection
│   ├── redis/          # Redis client
│   ├── kafka/          # Kafka producer/consumer
│   ├── firebase/       # Firebase services
│   └── utils/          # Utility functions
└── pkg/
    ├── auth/           # JWT authentication
    ├── response/       # API response helpers
    └── validation/     # Input validation
```

The main.go file for every microservice is outside the internal package.

Here are sample codes of how I have implemented vehicle-service

1. vehicle_model.go

```
package models

import (
"time"

"github.com/google/uuid"
"gorm.io/gorm"
)

type VehicleType string
type FuelType string
type TransmissionType string
type VehicleStatus string

const (
VehicleTypeCar VehicleType = "car"
VehicleTypeMotorcycle VehicleType = "motorcycle"
VehicleTypeTruck VehicleType = "truck"
VehicleTypeBus VehicleType = "bus"

FuelTypePetrol FuelType = "petrol"
FuelTypeDiesel FuelType = "diesel"
FuelTypeElectric FuelType = "electric"
FuelTypeHybrid FuelType = "hybrid"

TransmissionManual TransmissionType = "manual"
TransmissionAutomatic TransmissionType = "automatic"
TransmissionCVT TransmissionType = "cvt"

VehicleStatusActive VehicleStatus = "active"
VehicleStatusInactive VehicleStatus = "inactive"
VehicleStatusSold VehicleStatus = "sold"
)

// Customer represents minimal customer data synced from user service
type Customer struct {
ID string `json:"id" gorm:"type:uuid;primary_key"`
Email string `json:"email" gorm:"not null"`
Phone string `json:"phone" gorm:"not null"`
FirstName string `json:"first_name" gorm:"not null"`
LastName string `json:"last_name" gorm:"not null"`
```

```
Status string `json:"status" gorm:"not null"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
}

type VehicleMake struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"unique;not null"`
Country string `json:"country"`
LogoURL *string `json:"logo_url,omitempty"`
IsActive bool `json:"is_active" gorm:"default:true"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

Models []VehicleModel `json:"models" gorm:"foreignKey:MakeID"`
}

type VehicleModel struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
MakeID string `json:"make_id" gorm:"not null"`
Name string `json:"name" gorm:"not null"`
Description *string `json:"description,omitempty"` // ADDED: Description field
VehicleType VehicleType `json:"vehicle_type" gorm:"not null"`
StartYear int `json:"start_year"`
EndYear *int `json:"end_year,omitempty"`
IsActive bool `json:"is_active" gorm:"default:true"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

Make VehicleMake `json:"make" gorm:"foreignKey:MakeID"`
CustomerVehicles []CustomerVehicle `json:"customer_vehicles"
gorm:"foreignKey:ModelID"`
}

type CustomerVehicle struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
CustomerID string `json:"customer_id" gorm:"not null"`
MakeID string `json:"make_id" gorm:"not null"`
ModelID string `json:"model_id" gorm:"not null"`
RegistrationNo string `json:"registration_no" gorm:"unique;not null"`
YearOfManufacture int `json:"year_of_manufacture" gorm:"not null"`
Color string `json:"color"`
ChassisNumber *string `json:"chassis_number,omitempty"`
EngineNumber *string `json:"engine_number,omitempty"`
```

```
FuelType FuelType `json:"fuel_type" gorm:"not null"` // ADDED: FuelType field
Transmission TransmissionType `json:"transmission" gorm:"not null"` // ADDED:
Transmission field
EngineCapacity *float64 `json:"engine_capacity,omitempty"`
Mileage *int `json:"mileage,omitempty"`
PurchaseDate *time.Time `json:"purchase_date,omitempty"`
Status VehicleStatus `json:"status" gorm:"default:active"`
Notes *string `json:"notes,omitempty"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

Customer Customer `json:"customer" gorm:"foreignKey:CustomerID"`
Make VehicleMake `json:"make" gorm:"foreignKey:MakeID"`
Model VehicleModel `json:"model" gorm:"foreignKey:ModelID"`
}

type VehicleImage struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
ImageURL string `json:"image_url" gorm:"not null"`
ImageType string `json:"image_type"` // front, back, side, interior, engine, documents
IsPrimary bool `json:"is_primary" gorm:"default:false"`
CreatedAt time.Time `json:"created_at"`

Vehicle CustomerVehicle `json:"-" gorm:"foreignKey:VehicleID"`
}

func (c *Customer) BeforeCreate(tx *gorm.DB) error {
if c.ID == "" {
c.ID = uuid.New().String()
}
return nil
}

func (vm *VehicleMake) BeforeCreate(tx *gorm.DB) error {
if vm.ID == "" {
vm.ID = uuid.New().String()
}
return nil
}

func (vmodel *VehicleModel) BeforeCreate(tx *gorm.DB) error {
if vmodel.ID == "" {
vmodel.ID = uuid.New().String()
```

```
}
return nil
}

func (cv *CustomerVehicle) BeforeCreate(tx *gorm.DB) error {
if cv.ID == "" {
cv.ID = uuid.New().String()
}
return nil
}
```

2. routes.go

```
package routes

import (
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-
management/middleware"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/handlers"
"github.com/gin-gonic/gin"
swaggerFiles "github.com/swaggo/files"
ginSwagger "github.com/swaggo/gin-swagger"
)

func SetupRoutes(
r *gin.Engine,
vehicleHandler *handlers.VehicleHandler,
authMiddleware *middleware.AuthMiddleware,
) {
// Add CORS middleware
r.Use(middleware.CORSMiddleware())

// Add logging middleware
r.Use(middleware.LoggingMiddleware())

// Health check
r.GET("/health", func(c *gin.Context) {
c.JSON(200, gin.H{
"status": "healthy",
"service": "vehicle-service",
"timestamp": time.Now(),
```

```
})
})

// Swagger documentation
r.GET("/docs/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))

// API routes
api := r.Group("/api/v1")

// Public vehicle reference data (no auth required)
vehicles := api.Group("/vehicles")
{
vehicles.GET("/makes", vehicleHandler.GetAllMakes)
//vehicles.GET("/makes/:id", vehicleHandler.GetMakeByID)
//vehicles.GET("/models", vehicleHandler.GetAllModels)
vehicles.GET("/makes/:make_id/models", vehicleHandler.GetModelsByMake)
vehicles.GET("/registration/:registration", vehicleHandler.GetVehicleByRegistration)
}

// Protected routes
protected := api.Group("/")
protected.Use(authMiddleware.RequireAuth())
{
// Customer vehicle operations
protected.POST("/vehicles/customer", vehicleHandler.CreateCustomerVehicle)
//protected.GET("/vehicles/:id", vehicleHandler.GetVehicleByID)

// Customer-specific routes
customers := protected.Group("/customers")
{
customers.GET("/:customer_id/vehicles", vehicleHandler.GetCustomerVehicles)
}

// Admin only routes
admin := protected.Group("/admin")
admin.Use(authMiddleware.RequireRole("admin", "super_admin"))
{
admin.POST("/vehicles/makes", vehicleHandler.CreateMake)
admin.POST("/vehicles/models", vehicleHandler.CreateModel)
}
}
}
```

3 . vehicle_handlers.go

```
package handlers

import (
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/response"
"github.com/gin-gonic/gin"
"github.com/go-playground/validator/v10"
)

type VehicleHandler struct {
vehicleService *services.VehicleService
validator *validator.Validate
}

func NewVehicleHandler(vehicleService *services.VehicleService) *VehicleHandler {
return &VehicleHandler{
vehicleService: vehicleService,
validator: validator.New(),
}
}

// Make handlers
func (h *VehicleHandler) CreateMake(c *gin.Context) {
var req services.CreateMakeRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

make, err := h.vehicleService.CreateMake(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Vehicle make created successfully", make)
}
```

```
func (h *VehicleHandler) GetAllMakes(c *gin.Context) {
makes, err := h.vehicleService.GetAllMakes()
if err != nil {
response.InternalServerError(c, "Failed to retrieve vehicle makes")
return
}

response.Success(c, "Vehicle makes retrieved successfully", makes)
}

//func (h *VehicleHandler) GetMakeByID(c *gin.Context) {
// makeID := c.Param("id")
//
// make, err := h.vehicleService.GetMakeByID(makeID)
// if err != nil {
// response.NotFound(c, "Vehicle make not found")
// return
// }
//
// response.Success(c, "Vehicle make retrieved successfully", make)
//}

// Model handlers
func (h *VehicleHandler) CreateModel(c *gin.Context) {
var req services.CreateModelRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

model, err := h.vehicleService.CreateModel(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Vehicle model created successfully", model)
}
```

```
//func (h *VehicleHandler) GetAllModels(c *gin.Context) {
// models, err := h.vehicleService.GetAllModels()
// if err != nil {
// response.InternalServerError(c, "Failed to retrieve vehicle models")
// return
// }
//
// response.Success(c, "Vehicle models retrieved successfully", models)
//}

func (h *VehicleHandler) GetModelsByMake(c *gin.Context) {
makeID := c.Param("make_id")

models, err := h.vehicleService.GetModelsByMake(makeID)
if err != nil {
response.InternalServerError(c, "Failed to retrieve vehicle models")
return
}

response.Success(c, "Vehicle models retrieved successfully", models)
}

// Customer Vehicle handlers
func (h *VehicleHandler) CreateCustomerVehicle(c *gin.Context) {
var req services.CreateCustomerVehicleRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

// Authorization check for customers
userID := c.GetString("user_id")
userType := c.GetString("user_type")

if userType == "customer" && req.CustomerID != userID {
response.Forbidden(c, "Cannot create vehicle for another customer")
return
}
```

```
vehicle, err := h.vehicleService.CreateCustomerVehicle(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Customer vehicle created successfully", vehicle)
}

func (h *VehicleHandler) GetCustomerVehicles(c *gin.Context) {
customerID := c.Param("customer_id")

// Authorization check
userID := c.GetString("user_id")
userType := c.GetString("user_type")

if userType == "customer" && customerID != userID {
response.Forbidden(c, "Cannot access another customer's vehicles")
return
}

vehicles, err := h.vehicleService.GetCustomerVehicles(customerID)
if err != nil {
response.InternalServerError(c, "Failed to retrieve customer vehicles")
return
}

response.Success(c, "Customer vehicles retrieved successfully", vehicles)
}

//func (h *VehicleHandler) GetVehicleByID(c *gin.Context) {
// vehicleID := c.Param("id")
//
// vehicle, err := h.vehicleService.GetCustomerVehicles(vehicleID)
// if err != nil {
// response.NotFound(c, "Vehicle not found")
// return
// }
//
// // Authorization check
// userID := c.GetString("user_id")
// userType := c.GetString("user_type")
//
```

```
// if userType == "customer" && vehicle.CustomerID != userID {
// response.Forbidden(c, "Cannot access another customer's vehicle")
// return
// }
//
// response.Success(c, "Vehicle retrieved successfully", vehicle)
//}

func (h *VehicleHandler) GetVehicleByRegistration(c *gin.Context) {
registrationNumber := c.Param("registration")

vehicle, err := h.vehicleService.GetVehicleByRegistration(registrationNumber)
if err != nil {
response.NotFound(c, "Vehicle not found")
return
}

response.Success(c, "Vehicle retrieved successfully", vehicle)
}
```

4. vehicle_service.go

```
package services

import (
"fmt"
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/kafka"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/repositories"
"github.com/google/uuid"
)

type VehicleService struct {
vehicleRepo *repositories.VehicleRepository
kafkaProducer *kafka.Producer
}

func NewVehicleService(vehicleRepo *repositories.VehicleRepository, kafkaProducer
*kafka.Producer) *VehicleService {
return &VehicleService{
```

```
vehicleRepo: vehicleRepo,
kafkaProducer: kafkaProducer,
}
}

// Customer Vehicle operations
type CreateCustomerVehicleRequest struct {
CustomerID string `json:"customer_id" validate:"required"`
MakeID string `json:"make_id" validate:"required"`
ModelID string `json:"model_id" validate:"required"`
RegistrationNo string `json:"registration_no" validate:"required"`
YearOfManufacture int `json:"year_of_manufacture"
validate:"required,min=1900,max=2030"`
Color string `json:"color"`
ChassisNumber *string `json:"chassis_number,omitempty"`
EngineNumber *string `json:"engine_number,omitempty"`
FuelType models.FuelType `json:"fuel_type" validate:"required"` // FIXED: Now
available in models
Transmission models.TransmissionType `json:"transmission" validate:"required"` //
FIXED: Now available in models
EngineCapacity *float64 `json:"engine_capacity,omitempty"`
Mileage *int `json:"mileage,omitempty"`
PurchaseDate *time.Time `json:"purchase_date,omitempty"`
Notes *string `json:"notes,omitempty"`
}

func (s *VehicleService) CreateCustomerVehicle(req CreateCustomerVehicleRequest)
(*models.CustomerVehicle, error) {
// Verify customer exists
_, err := s.vehicleRepo.GetCustomerByID(req.CustomerID)
if err != nil {
return nil, fmt.Errorf("customer not found: %v", err)
}

// Verify make exists
_, err = s.vehicleRepo.GetMakeByID(req.MakeID)
if err != nil {
return nil, fmt.Errorf("vehicle make not found: %v", err)
}

// Verify model exists
model, err := s.vehicleRepo.GetModelByID(req.ModelID)
if err != nil {
return nil, fmt.Errorf("vehicle model not found: %v", err)
```

```
}

// Verify model belongs to make
if model.MakeID != req.MakeID {
return nil, fmt.Errorf("model does not belong to the specified make")
}

// Check if registration number already exists
existing, err := s.vehicleRepo.GetVehicleByRegistration(req.RegistrationNo)
if err == nil && existing != nil {
return nil, fmt.Errorf("vehicle with registration number %s already exists",
req.RegistrationNo)
}

// Create vehicle
vehicle := &models.CustomerVehicle{
CustomerID: req.CustomerID,
MakeID: req.MakeID,
ModelID: req.ModelID,
RegistrationNo: req.RegistrationNo,
YearOfManufacture: req.YearOfManufacture,
Color: req.Color,
ChassisNumber: req.ChassisNumber,
EngineNumber: req.EngineNumber,
FuelType: req.FuelType, // FIXED: Now works
Transmission: req.Transmission, // FIXED: Now works
EngineCapacity: req.EngineCapacity,
Mileage: req.Mileage,
PurchaseDate: req.PurchaseDate,
Notes: req.Notes,
Status: models.VehicleStatusActive,
}

if err := s.vehicleRepo.CreateCustomerVehicle(vehicle); err != nil {
return nil, fmt.Errorf("failed to create vehicle: %v", err)
}

// Publish vehicle created event
s.publishVehicleEvent("vehicle.created", vehicle)

return vehicle, nil
}

// Event handlers for customer sync
```

```
func (s *VehicleService) HandleCustomerCreated(event kafka.Event) error {
customerData := event.Data

customer := &models.Customer{ // FIXED: Now using local Customer model
ID: customerData["user_id"].(string),
Email: customerData["email"].(string),
Phone: customerData["phone"].(string),
FirstName: customerData["first_name"].(string),
LastName: customerData["last_name"].(string),
Status: "pending_verification",
}

if err := s.vehicleRepo.CreateOrUpdateCustomer(customer); err != nil {
return fmt.Errorf("failed to create customer in vehicle service: %v", err)
}

return nil
}

func (s *VehicleService) HandleCustomerVerified(event kafka.Event) error {
customerID := event.Data["user_id"].(string)

customer, err := s.vehicleRepo.GetCustomerByID(customerID)
if err != nil {
return fmt.Errorf("customer not found in vehicle service: %v", err)
}

customer.Status = "active"
return s.vehicleRepo.CreateOrUpdateCustomer(customer)
}

// Get methods
func (s *VehicleService) GetCustomerVehicles(customerID string)
([]models.CustomerVehicle, error) {
return s.vehicleRepo.GetCustomerVehicles(customerID)
}

func (s *VehicleService) GetVehicleByID(vehicleID string) (*models.CustomerVehicle,
error) {
return s.vehicleRepo.GetVehicleByID(vehicleID)
}

func (s *VehicleService) GetVehicleByRegistration(regNo string)
(*models.CustomerVehicle, error) {
```

```
return s.vehicleRepo.GetVehicleByRegistration(regNo)
}

// Vehicle Make operations
type CreateMakeRequest struct {
Name string `json:"name" validate:"required"`
Country string `json:"country"`
LogoURL *string `json:"logo_url,omitempty"`
}

func (s *VehicleService) CreateMake(req CreateMakeRequest) (*models.VehicleMake,
error) {
make := &models.VehicleMake{
Name: req.Name,
Country: req.Country,
LogoURL: req.LogoURL,
}

if err := s.vehicleRepo.CreateMake(make); err != nil {
return nil, fmt.Errorf("failed to create make: %v", err)
}

return make, nil
}

func (s *VehicleService) GetAllMakes() ([]models.VehicleMake, error) {
return s.vehicleRepo.GetAllMakes()
}

// Vehicle Model operations
type CreateModelRequest struct {
MakeID string `json:"make_id" validate:"required"`
Name string `json:"name" validate:"required"`
Description *string `json:"description,omitempty"` // FIXED: Now available
VehicleType models.VehicleType `json:"vehicle_type" validate:"required"`
StartYear int `json:"start_year" validate:"required"`
EndYear *int `json:"end_year,omitempty"`
}

func (s *VehicleService) CreateModel(req CreateModelRequest)
(*models.VehicleModel, error) {
// Verify make exists
_, err := s.vehicleRepo.GetMakeByID(req.MakeID)
if err != nil {
```

```
return nil, fmt.Errorf("vehicle make not found: %v", err)
}

model := &models.VehicleModel{
MakeID: req.MakeID,
Name: req.Name,
Description: req.Description, // FIXED: Now works
VehicleType: req.VehicleType,
StartYear: req.StartYear,
EndYear: req.EndYear,
}

if err := s.vehicleRepo.CreateModel(model); err != nil {
return nil, fmt.Errorf("failed to create model: %v", err)
}

return model, nil
}

func (s *VehicleService) GetModelsByMake(makeID string) ([]models.VehicleModel,
error) {
return s.vehicleRepo.GetModelsByMake(makeID)
}

// Utility methods
func (s *VehicleService) SearchVehicles(query string, limit, offset int)
([]models.CustomerVehicle, error) {
return s.vehicleRepo.SearchVehicles(query, limit, offset)
}

func (s *VehicleService) publishVehicleEvent(eventType string, vehicle
*models.CustomerVehicle) {
if s.kafkaProducer == nil {
return
}

event := kafka.Event{
ID: uuid.New().String(),
Type: eventType,
Source: "vehicle-service",
Data: map[string]interface{}{
"vehicle_id": vehicle.ID,
"customer_id": vehicle.CustomerID,
"registration_no": vehicle.RegistrationNo,
```

```
"make_id": vehicle.MakeID,
"model_id": vehicle.ModelID,
"year": vehicle.YearOfManufacture,
"fuel_type": string(vehicle.FuelType),
"transmission": string(vehicle.Transmission),
},
Timestamp: time.Now().Unix(),
Version: "1.0",
}

s.kafkaProducer.PublishEvent("vehicle_events", event)
}
```

5. vehicle_repositories.go

```
package repositories

import (
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/models"
"gorm.io/gorm"
)

type VehicleRepository struct {
db *gorm.DB
}

func NewVehicleRepository(db *gorm.DB) *VehicleRepository {
return &VehicleRepository{db: db}
}

// Customer operations (synced from user service)
func (r *VehicleRepository) CreateOrUpdateCustomer(customer *models.Customer)
error {
var existingCustomer models.Customer
err := r.db.Where("id = ?", customer.ID).First(&existingCustomer).Error

if err == gorm.ErrRecordNotFound {
return r.db.Create(customer).Error
}

return r.db.Save(customer).Error
}
```

```
func (r *VehicleRepository) GetCustomerByID(customerID string) (*models.Customer,
error) {
var customer models.Customer
err := r.db.First(&customer, "id = ?", customerID).Error
return &customer, err
}

// Vehicle Make operations
func (r *VehicleRepository) CreateMake(make *models.VehicleMake) error {
return r.db.Create(make).Error
}

func (r *VehicleRepository) GetAllMakes() ([]models.VehicleMake, error) {
var makes []models.VehicleMake
err := r.db.Where("is_active = ?", true).Order("name").Find(&makes).Error
return makes, err
}

func (r *VehicleRepository) GetMakeByID(makeID string) (*models.VehicleMake,
error) {
var make models.VehicleMake
err := r.db.Preload("Models").First(&make, "id = ?", makeID).Error
return &make, err
}

func (r *VehicleRepository) UpdateMake(make *models.VehicleMake) error {
return r.db.Save(make).Error
}

// Vehicle Model operations
func (r *VehicleRepository) CreateModel(model *models.VehicleModel) error {
return r.db.Create(model).Error
}

func (r *VehicleRepository) GetModelsByMake(makeID string)
([]models.VehicleModel, error) {
var models []models.VehicleModel
err := r.db.Where("make_id = ? AND is_active = ?", makeID,
true).Order("name").Find(&models).Error
return models, err
}

func (r *VehicleRepository) GetModelByID(modelID string) (*models.VehicleModel,
error) {
```

```
var model models.VehicleModel
err := r.db.Preload("Make").First(&model, "id = ?", modelID).Error
return &model, err
}

func (r *VehicleRepository) UpdateModel(model *models.VehicleModel) error {
return r.db.Save(model).Error
}

// Customer Vehicle operations
func (r *VehicleRepository) CreateCustomerVehicle(vehicle *models.CustomerVehicle)
error {
return r.db.Create(vehicle).Error
}

func (r *VehicleRepository) GetCustomerVehicles(customerID string)
([]models.CustomerVehicle, error) {
var vehicles []models.CustomerVehicle
err := r.db.Preload("Make").Preload("Model").Where("customer_id = ?",
customerID).Find(&vehicles).Error
return vehicles, err
}

func (r *VehicleRepository) GetVehicleByID(vehicleID string)
(*models.CustomerVehicle, error) {
var vehicle models.CustomerVehicle
err := r.db.Preload("Customer").Preload("Make").Preload("Model").First(&vehicle, "id
= ?", vehicleID).Error
return &vehicle, err
}

func (r *VehicleRepository) GetVehicleByRegistration(regNo string)
(*models.CustomerVehicle, error) {
var vehicle models.CustomerVehicle
err := r.db.Preload("Customer").Preload("Make").Preload("Model").First(&vehicle,
"registration_no = ?", regNo).Error
return &vehicle, err
}

func (r *VehicleRepository) UpdateCustomerVehicle(vehicle
*models.CustomerVehicle) error {
return r.db.Save(vehicle).Error
}
```

```
func (r *VehicleRepository) DeleteCustomerVehicle(vehicleID string) error {
return r.db.Delete(&models.CustomerVehicle{}, "id = ?", vehicleID).Error
}

// Vehicle Image operations
func (r *VehicleRepository) CreateVehicleImage(image *models.VehicleImage) error {
return r.db.Create(image).Error
}

func (r *VehicleRepository) GetVehicleImages(vehicleID string)
([]models.VehicleImage, error) {
var images []models.VehicleImage
err := r.db.Where("vehicle_id = ?", vehicleID).Order("is_primary DESC, created_at
ASC").Find(&images).Error
return images, err
}

func (r *VehicleRepository) DeleteVehicleImage(imageID string) error {
return r.db.Delete(&models.VehicleImage{}, "id = ?", imageID).Error
}

// Search operations
func (r *VehicleRepository) SearchVehicles(query string, limit, offset int)
([]models.CustomerVehicle, error) {
var vehicles []models.CustomerVehicle
err := r.db.Preload("Customer").Preload("Make").Preload("Model").
Where("registration_no ILIKE ? OR chassis_number ILIKE ? OR engine_number
ILIKE ?",
"%"+query+"%", "%"+query+"%", "%"+query+"%").
Limit(limit).Offset(offset).Find(&vehicles).Error
return vehicles, err
}
```

6. main.go

```
package main

import (
"context"
"log"
"net/http"
"os"
"os/signal"
"syscall"
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/database"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/kafka"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/redis"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/config"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/middleware"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/handlers"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/repositories"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/routes"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/auth"
"github.com/gin-gonic/gin"
"github.com/joho/godotenv"
"gorm.io/gorm"
)

// @title AA Core System - Vehicle Service API
// @version 1.0
// @description Vehicle management service for AA Core System
// @contact.name API Support
// @contact.email support@aa.co.ke
// @host localhost:8082
// @BasePath /api/v1
// @securityDefinitions.apikey BearerAuth
```

```
// @in header
// @name Authorization
func main() {
// Load environment variables
if err := godotenv.Load(); err != nil {
log.Println("No .env file found, using system environment variables")
}

// Load configuration
cfg, err := config.LoadConfig("./configs")
if err != nil {
log.Fatalf("Failed to load config: %v", err)
}

// Set Gin mode
gin.SetMode(cfg.Server.Mode)

// Initialize database
dbConfig := database.Config{
Host: cfg.Database.Host,
Port: cfg.Database.Port,
User: cfg.Database.User,
Password: cfg.Database.Password,
DBName: "aa_vehicle_service", // Separate database
SSLMode: cfg.Database.SSLMode,
MaxOpenConns: cfg.Database.MaxOpenConns,
MaxIdleConns: cfg.Database.MaxIdleConns,
ConnMaxLifetime: cfg.Database.ConnMaxLifetime,
}

db, err := database.NewPostgresConnection(dbConfig)
if err != nil {
log.Fatalf("Failed to connect to database: %v", err)
}

// Run migrations
if err := runMigrations(db); err != nil {
log.Fatalf("Failed to run migrations: %v", err)
}

// Initialize Redis
redisConfig := redis.Config{
Host: cfg.Redis.Host,
Port: cfg.Redis.Port,
```

```
Password: cfg.Redis.Password,
DB: 2, // Different Redis DB for vehicle service
PoolSize: cfg.Redis.PoolSize,
}

redisClient, err := redis.NewRedisClient(redisConfig)
if err != nil {
log.Fatalf("Failed to connect to Redis: %v", err)
}

// Initialize Kafka
kafkaConfig := kafka.Config{
Brokers: cfg.Kafka.Brokers,
Topics: map[string]string{
"user_events": "user-events",
"vehicle_events": "vehicle-events",
},
}

kafkaProducer := kafka.NewProducer(kafkaConfig)
defer kafkaProducer.Close()

// Initialize JWT manager
jwtManager := auth.NewJWTManager(
cfg.JWT.Secret,
cfg.JWT.AccessTokenDuration,
cfg.JWT.RefreshTokenDuration,
)

// Initialize repositories
vehicleRepo := repositories.NewVehicleRepository(db)

// Initialize services
vehicleService := services.NewVehicleService(vehicleRepo, kafkaProducer)

// Initialize middleware
authMiddleware := middleware.NewAuthMiddleware(jwtManager, nil, redisClient)

// Initialize handlers
vehicleHandler := handlers.NewVehicleHandler(vehicleService)

// Initialize Gin router
router := gin.New()
router.Use(gin.Recovery())
```

```
// Setup routes
routes.SetupRoutes(router, vehicleHandler, authMiddleware)

// Start Kafka consumers in background
go startKafkaConsumers(kafkaConfig, vehicleService)

// Seed initial data
go func() {
time.Sleep(5 * time.Second) // Wait for service to be ready
if err := seedInitialData(vehicleService); err != nil {
log.Printf("Failed to seed initial data: %v", err)
}
}()

// Create HTTP server
srv := &http.Server{
Addr: ":8082", // Different port for vehicle service
Handler: router,
ReadTimeout: cfg.Server.ReadTimeout,
WriteTimeout: cfg.Server.WriteTimeout,
}

// Start server in a goroutine
go func() {
log.Printf("Starting vehicle service on port 8082")
if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
log.Fatalf("Failed to start server: %v", err)
}
}()

// Wait for interrupt signal to gracefully shutdown the server
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

log.Println("Shutting down vehicle service...")

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
log.Fatalf("Server forced to shutdown: %v", err)
}
```

```
log.Println("Vehicle service shutdown complete")
}

func runMigrations(db *gorm.DB) error {
log.Println("Running vehicle service migrations...")

err := db.AutoMigrate(
&models.Customer{},
&models.VehicleMake{},
&models.VehicleModel{},
&models.CustomerVehicle{},
)

if err != nil {
return err
}

log.Println("Vehicle service migrations completed successfully")
return nil
}

func startKafkaConsumers(kafkaConfig kafka.Config, vehicleService
*services.VehicleService) {
consumer := kafka.NewConsumer(kafkaConfig, "vehicle-service")

// Handle user events from user-management service
go func() {
err := consumer.ConsumeEvents("user_events", func(event kafka.Event) error {
switch event.Type {
case "user.registered":
return vehicleService.HandleCustomerCreated(event)
case "user.email_verified":
return vehicleService.HandleCustomerVerified(event)
default:
log.Printf("Unhandled event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume user events: %v", err)
}
}()
}
```

```
func seedInitialData(vehicleService *services.VehicleService) error {
log.Println("Seeding initial vehicle data...")

// Popular vehicle makes
makes := []services.CreateMakeRequest{
{Name: "Toyota", Country: "Japanese"},
{Name: "Honda", Country: "Japanese"},
{Name: "Nissan", Country: "Japanese"},
{Name: "Mazda", Country: "Japanese"},
{Name: "Subaru", Country: "Japanese"},
{Name: "Mitsubishi", Country: "Japanese"},
{Name: "Suzuki", Country: "Japanese"},
{Name: "Mercedes-Benz", Country: "German"},
{Name: "BMW", Country: "German"},
{Name: "Audi", Country: "German"},
{Name: "Volkswagen", Country: "German"},
{Name: "Ford", Country: "American"},
{Name: "Chevrolet", Country: "American"},
{Name: "Hyundai", Country: "Korean"},
{Name: "Kia", Country: "Korean"},
{Name: "Peugeot", Country: "French"},
{Name: "Renault", Country: "French"},
{Name: "Volvo", Country: "Swedish"},
{Name: "Land Rover", Country: "British"},
{Name: "Isuzu", Country: "Japanese"},
}

// Create makes
for _, makeReq := range makes {
if _, err := vehicleService.CreateMake(makeReq); err != nil {
continue
}
}

log.Println("Initial vehicle data seeded successfully")
return nil
}
```